

Pipeline Processing in Computer Architecture

Computer Architecture and Organization Notes

June 27, 2025

Contents

1	Introduction to Pipeline Processing	3
1.1	What is Pipelining?	3
1.1.1	Real-World Analogy: Assembly Line	3
1.2	Applications of Pipelining	3
2	Pipeline Processor Architecture	4
2.1	Basic Structure	4
2.1.1	Pipeline Design Fundamentals	4
2.1.2	Key Components	4
2.2	Pipeline Architecture Diagram	5
2.3	Pipeline Operation Principles	5
2.3.1	Working Mechanism	5
2.3.2	Pipeline Execution Table	5
2.3.3	Concurrent Processing	6
2.3.4	Water Pipe Analogy	6
3	Pipeline Performance Analysis	6
3.1	Key Performance Metrics	6
3.1.1	Timing Parameters	6
3.1.2	Capacity and Efficiency	6
3.2	Speedup Calculation	7
4	Instruction Pipeline	7
4.1	Instruction Execution Stages	7
4.2	Instruction Pipeline Diagram	7
4.3	Instruction Pipeline Example	7
4.4	Branch Instruction Challenge	8
4.4.1	Branch Handling Process	8

5	Pipeline Design Principles	8
5.1	Design Methodology	8
5.1.1	Algorithm Decomposition	8
5.1.2	Hardware Implementation	9
5.2	Design Challenges	9
5.2.1	Pipeline Hazards	9
5.2.2	Balancing Considerations	9
6	Advanced Pipeline Concepts	9
6.1	Pipeline Efficiency	9
6.1.1	Ideal vs. Real Performance	9
6.1.2	Throughput Optimization	9
6.2	Pipeline Variations	10
6.2.1	Linear vs. Non-linear Pipelines	10
6.2.2	Synchronous vs. Asynchronous	10
7	Practical Applications	10
7.1	Processor Pipelines	10
7.2	Specialized Pipelines	10
8	Summary and Key Takeaways	10
8.1	Core Concepts	10
8.2	Design Principles	11
8.3	Pipeline Architecture Components	11
8.4	Performance Formula Summary	11
8.5	Key Pipeline Hazards	11
8.6	Real-World Applications	12
8.7	Design Trade-offs	12
9	Pipeline Hazards	12
9.1	Types of Pipeline Hazards	12
9.1.1	1. Structural Hazards	12
9.1.2	2. Data Hazards	13
9.1.3	3. Control Hazards	13
10	Performance Evaluation Factors	14
10.1	Key Performance Metrics	14
10.1.1	1. Latency	14
10.1.2	2. Efficiency	14
10.1.3	3. Throughput	14
10.2	Performance Comparison	15
11	Benefits and Disadvantages of Pipelining	15
11.1	Benefits of Pipelining	15
11.2	Disadvantages of Pipelining	15
11.3	When to Use Pipelining	16

1 Introduction to Pipeline Processing

1.1 What is Pipelining?

Definition: Pipelining is a general technique for increasing processor throughput without requiring large amounts of extra hardware by overlapping the execution of multiple instructions.

1.1.1 Real-World Analogy: Assembly Line

Think of pipelining like an automobile assembly line:

- **Traditional Manufacturing:** One worker builds an entire car from start to finish
- **Assembly Line (Pipeline):** Multiple workers, each specializing in one task:
 - Worker 1: Installs engine
 - Worker 2: Attaches wheels
 - Worker 3: Paints the car
 - Worker 4: Installs interior

Key Insight: While it still takes the same total time to complete one car, multiple cars can be in production simultaneously, dramatically increasing throughput.

1.2 Applications of Pipelining

Pipelining is applied in two main areas:

1. **Datapath Units:** Complex arithmetic operations
 - Multipliers
 - Floating-point adders
 - Division units
2. **Instruction Processing:** Overall processor throughput
 - Instruction fetch
 - Instruction decode
 - Execute
 - Memory access
 - Write back

2 Pipeline Processor Architecture

2.1 Basic Structure

Pipeline Processor: A sequence of m data-processing circuits (stages/segments) that collectively perform a single operation on a stream of data operands passing through them.

Core Concept: A pipeline processor improves processing speed by dividing instruction execution into separate stages. Each stage handles a specific part of the instruction, allowing multiple instructions to be processed simultaneously at different stages.

2.1.1 Pipeline Design Fundamentals

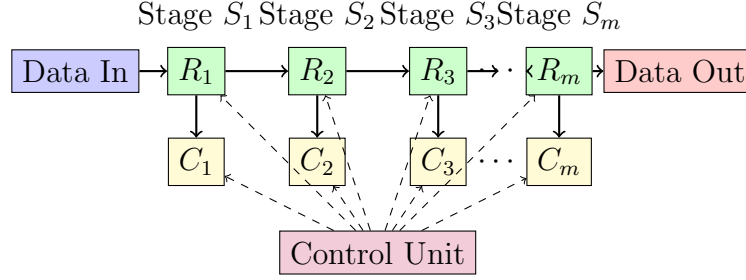
A pipelined processor has the following structural characteristics:

- **Two Ends:** Input end (where data enters) and output end (where results emerge)
- **Multiple Stages:** Between the ends, there are multiple processing stages
- **Sequential Connection:** Output of one stage connects to input of next stage
- **Stage-Specific Operations:** Each stage performs a specific operation
- **Interface Registers:** Hold intermediate results between stages (also called latches or buffers)
- **Common Clock:** All stages and registers controlled by a single clock signal

2.1.2 Key Components

- **Data In:** Input data that enters the pipeline
- **Stages ($S_1, S_2, \dots, S_m$):** Individual processing units, each handling specific operations
- **Pipeline Registers ($R_1, R_2, \dots, R_m$):** Temporary storage between stages for smooth data transfer and isolation
- **Computation Units ($C_1, C_2, \dots, C_m$):** Perform actual processing (arithmetic, logical operations)
- **Control Unit:** Manages timing and control signals, ensures synchronized operation
- **Data Out:** Final output after processing through all pipeline stages

2.2 Pipeline Architecture Diagram



Legend:

- R_i : Pipeline Registers (Latches/Buffers)
- C_i : Computation Units
- **Control Unit**: Timing and synchronization
- Dashed lines: Control and clock signals

2.3 Pipeline Operation Principles

2.3.1 Working Mechanism

The pipeline operates in a synchronized manner:

1. **Clock Cycle 1:** Data enters Stage S_1 (Register $R_1 \rightarrow$ Computation Unit C_1)
2. **Clock Cycle 2:** Previous data moves to Stage S_2 , while new data enters Stage S_1
3. **Continuous Process:** Each stage simultaneously processes different data
4. **Final Output:** Results emerge after passing through all stages

2.3.2 Pipeline Execution Table

Clock Cycle	Stage S_1 ($R_1 \beta C_1$)	Stage S_2 ($R_2 \beta C_2$)	Stage S_3 ($R_3 \beta C_3$)	...	Stage S_m ($R_m \beta C_m$)
1	Data A	-	-	...	-
2	Data B	Data A	-	...	-
3	Data C	Data B	Data A	...	-
4	Data D	Data C	Data B	...	-
5	Data E	Data D	Data C	...	Data A
...	...	...	...	...	...

Key Observation: After the pipeline fills up (after m clock cycles), a new result is produced every clock cycle.

2.3.3 Concurrent Processing

In each clock period:

1. Every stage transfers previous results to the next stage
2. Every stage computes a new set of results
3. Multiple operations execute simultaneously in different stages

2.3.4 Water Pipe Analogy

Imagine a water treatment plant with multiple purification stages:

- **Stage 1:** Filtration
- **Stage 2:** Chemical treatment
- **Stage 3:** UV sterilization
- **Stage 4:** Final testing

Water flows continuously through all stages. While Stage 4 is testing batch A, Stage 3 is sterilizing batch B, Stage 2 is treating batch C, and Stage 1 is filtering batch D.

3 Pipeline Performance Analysis

3.1 Key Performance Metrics

3.1.1 Timing Parameters

- **Clock Period (T):** Time each stage takes to complete its operation
- **Pipeline Latency:** mT (where m = number of stages)
- **Throughput:** $\frac{1}{T}$ operations per second
- **CPI (Cycles Per Instruction):** 1 (in ideal conditions)

3.1.2 Capacity and Efficiency

An m -stage pipeline can simultaneously process up to m independent data sets, with a new result emerging every clock cycle once the pipeline is full.

3.2 Speedup Calculation

For N operations in an m -stage pipeline:

$$\text{Sequential Time} = N \times mT \quad (1)$$

$$\text{Pipeline Time} = (N + m - 1)T \quad (2)$$

$$\text{Speedup} = \frac{N \times mT}{(N + m - 1)T} = \frac{Nm}{N + m - 1} \quad (3)$$

For large N : Speedup $\approx m$

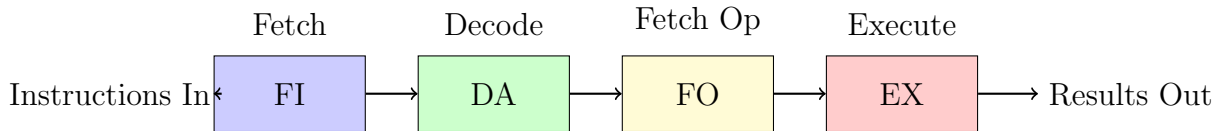
4 Instruction Pipeline

4.1 Instruction Execution Stages

The pipeline is most efficient when the instruction cycle is divided into segments of equal duration. In general, a computer processes each instruction through the following sequence:

1. **FI (Fetch Instruction)**: Fetches the instruction into the instruction register
2. **DA (Decode Address)**: Decodes the instruction to determine the opcode and addressing mode
3. **FO (Fetch Operands)**: Fetches the operands into the data register
4. **EX (Execute)**: Executes the specified operation and stores the result

4.2 Instruction Pipeline Diagram



4.3 Instruction Pipeline Example

Clock Cycle	FI (Fetch)	DA (Decode)	FO (Fetch Op)	EX (Execute)
1	Instruction 1	-	-	-
2	Instruction 2	Instruction 1	-	-
3	Instruction 3	Instruction 2	Instruction 1	-
4	Instruction 4	Instruction 3	Instruction 2	Instruction 1
5	Instruction 5	Instruction 4	Instruction 3	Instruction 2
6	Instruction 6	Instruction 5	Instruction 4	Instruction 3

Key Observations:

- After cycle 4, one instruction completes execution every cycle
- Four instructions are being processed simultaneously in different stages
- This demonstrates **instruction-level parallelism**

4.4 Branch Instruction Challenge

Control Hazard Example: When the third instruction is a branch instruction, pipeline complications arise.

4.4.1 Branch Handling Process

Consider this scenario where Instruction 3 is a branch:

1. **Cycle 3:** Instruction 3 (branch) is in Decode stage
2. **Cycle 3:** Instruction 4 is fetched based on sequential assumption
3. **Problem:** Branch might alter execution flow, making Instruction 4 potentially incorrect
4. **Solution:** Instruction 4 is held or discarded until branch is resolved
5. **Cycle 4:** Branch outcome determined in Execute stage:
 - If branch taken: Re-fetch correct target instruction
 - If branch not taken: Continue with Instruction 4

Impact: This demonstrates a **control hazard**, requiring pipeline stalls or corrective actions, which reduces overall pipeline efficiency.

5 Pipeline Design Principles

5.1 Design Methodology

5.1.1 Algorithm Decomposition

1. **Identify Sequential Steps:** Break complex operation into simpler sub-operations
2. **Balance Stages:** Ensure each stage has roughly equal execution time
3. **Minimize Dependencies:** Reduce data dependencies between stages
4. **Optimize Critical Path:** Identify and optimize the slowest stage

5.1.2 Hardware Implementation

- **Buffer Registers:** Fast storage between stages
- **Clocked Design:** Synchronous operation using global clock
- **Pipeline Registers:** Hold intermediate results
- **Control Logic:** Manage data flow and synchronization

5.2 Design Challenges

5.2.1 Pipeline Hazards

1. **Structural Hazards:** Resource conflicts
2. **Data Hazards:** Data dependencies
3. **Control Hazards:** Branch instructions

5.2.2 Balancing Considerations

- **Stage Complexity:** Each stage should perform similar amount of work
- **Critical Path:** Pipeline speed limited by slowest stage
- **Register Overhead:** Additional storage requirements
- **Control Complexity:** Managing pipeline flow

6 Advanced Pipeline Concepts

6.1 Pipeline Efficiency

6.1.1 Ideal vs. Real Performance

$$\text{Ideal Speedup} = m \tag{4}$$

$$\text{Real Speedup} = \frac{m}{1 + \text{Hazard Penalty}} \tag{5}$$

6.1.2 Throughput Optimization

- **Increase Pipeline Depth:** More stages, higher throughput
- **Parallel Pipelines:** Multiple pipelines for different operations
- **Superpipelining:** Very deep pipelines with short clock periods
- **Superscalar:** Multiple pipelines executing simultaneously

6.2 Pipeline Variations

6.2.1 Linear vs. Non-linear Pipelines

- **Linear Pipeline:** Sequential stages, no feedback
- **Non-linear Pipeline:** Feedback loops, more complex control

6.2.2 Synchronous vs. Asynchronous

- **Synchronous:** Global clock, easier control
- **Asynchronous:** Handshaking, better performance potential

7 Practical Applications

7.1 Processor Pipelines

- **RISC Processors:** 5-stage pipeline (IF, ID, EX, MEM, WB)
- **CISC Processors:** Variable-length pipelines
- **Modern Processors:** 10-20+ stage pipelines

7.2 Specialized Pipelines

- **Graphics Processing:** Vertex and pixel pipelines
- **Digital Signal Processing:** Filter pipelines
- **Network Processing:** Packet processing pipelines

8 Summary and Key Takeaways

8.1 Core Concepts

1. **Pipelining increases throughput** without proportional hardware increase
2. **Latency remains the same** but throughput improves significantly
3. **Speedup approaches** the number of pipeline stages for large workloads
4. **Design requires careful balancing** of stage complexity and timing
5. **Pipeline hazards** can significantly impact performance and must be managed
6. **Instruction-level parallelism** is achieved through overlapping execution stages

8.2 Design Principles

- Break complex operations into balanced stages
- Use fast buffer registers between stages
- Minimize pipeline hazards and dependencies
- Optimize the critical path (slowest stage)
- Implement effective hazard detection and resolution mechanisms
- Consider branch prediction for control hazards
- Balance pipeline depth with complexity and power consumption

8.3 Pipeline Architecture Components

- **Stages (S_i):** Individual processing units performing specific operations
- **Pipeline Registers (R_i):** Interface storage between stages (latches/buffers)
- **Computation Units (C_i):** Actual processing elements (ALU, etc.)
- **Control Unit:** Manages timing, synchronization, and hazard resolution
- **Common Clock:** Synchronizes all pipeline operations

8.4 Performance Formula Summary

$$\text{Pipeline Latency} = mT \tag{6}$$

$$\text{Pipeline Throughput} = \frac{1}{T} \text{ (ideal)} \tag{7}$$

$$\text{Real Throughput} = \frac{1}{T \times (1 + \text{Stall CPI})} \tag{8}$$

$$\text{Speedup for } N \text{ operations} = \frac{Nm}{N + m - 1} \tag{9}$$

$$\text{Asymptotic Speedup} \approx m \text{ (for large } N) \tag{10}$$

$$\text{Efficiency} = \frac{\text{Useful work time}}{\text{Total pipeline time}} \tag{11}$$

8.5 Key Pipeline Hazards

1. **Structural Hazards:** Resource conflicts between pipeline stages
2. **Data Hazards:** Dependencies between instructions (RAW, WAR, WAW, RAR)
3. **Control Hazards:** Branch instructions disrupting sequential flow

8.6 Real-World Applications

- **Processor Design:** Modern CPUs use deep pipelines (10-20+ stages)
- **Graphics Processing:** GPU pipelines for parallel graphics operations
- **Digital Signal Processing:** Filter pipelines for real-time processing
- **Network Processing:** Packet processing pipelines in routers
- **Arithmetic Units:** Floating-point operation pipelines

8.7 Design Trade-offs

Aspect	Benefit	Cost
Deeper Pipeline	Higher throughput	Higher latency, more hazards
More Stages	Better resource utilization	Complex control logic
Hazard Resolution	Better performance	Additional hardware
Branch Prediction	Reduced stalls	Increased complexity

9 Pipeline Hazards

Pipeline Hazards: Problems that occur during pipelining which can stall the pipeline or flush it completely. Not all instructions are suitable for pipelining.

9.1 Types of Pipeline Hazards

9.1.1 1. Structural Hazards

Definition: Arise due to resource conflicts in the pipeline when more than one instruction tries to access the same resource in the same cycle.

Examples of Resources:

- Registers
- Memory units
- Arithmetic Logic Unit (ALU)
- Functional units

Solution Approaches:

- Resource duplication
- Pipeline stalling
- Resource scheduling

9.1.2 2. Data Hazards

Definition: Occur when an instruction depends on the result of a previous instruction that has not yet been computed.

Four Types of Data Dependencies:

1. Read After Write (RAW) - True Dependency:

- Instruction I_2 reads a register that I_1 writes
- Most common and problematic
- Example: ADD R1, R2, R3 followed by SUB R4, R1, R5

2. Write After Read (WAR) - Anti-dependency:

- Instruction I_2 writes a register that I_1 reads
- Less common in simple pipelines
- Example: ADD R1, R2, R3 followed by SUB R2, R4, R5

3. Write After Write (WAW) - Output Dependency:

- Both instructions write to the same register
- Final value must be from the later instruction
- Example: ADD R1, R2, R3 followed by SUB R1, R4, R5

4. Read After Read (RAR) - No Dependency:

- Both instructions read the same register
- No hazard, multiple reads are safe
- Example: ADD R1, R2, R3 followed by SUB R4, R2, R5

9.1.3 3. Control Hazards

Definition: Occur during the execution of control transfer instructions such as BRANCH, CALL, JMP, etc.

Problem: The processor doesn't know the target address when it needs to insert the new instruction into the pipeline, leading to unwanted instructions being fed to the pipeline.

Common Control Instructions:

- Conditional branches
- Unconditional jumps
- Function calls
- Returns

Solution Techniques:

- Branch prediction
- Delayed branching
- Pipeline flushing
- Branch target buffers

10 Performance Evaluation Factors

10.1 Key Performance Metrics

10.1.1 1. Latency

- **Definition:** Time taken for a single instruction to complete its execution
- **Formula:** Latency = mT (where m = number of stages, T = clock period)
- **Impact:** Lower latency indicates better performance for individual instructions
- **Trade-off:** Increasing pipeline depth increases latency but can improve throughput

10.1.2 2. Efficiency

- **Definition:** Measures how effectively pipeline stages are utilized
- **Formula:** Efficiency = $\frac{\text{Useful work time}}{\text{Total pipeline time}}$
- **Factors affecting efficiency:**
 - Pipeline stalls due to hazards
 - Unbalanced stage execution times
 - Branch mispredictions

10.1.3 3. Throughput

- **Definition:** Number of instructions completed per unit of time
- **Formula:** Throughput = $\frac{1}{T}$ (in ideal conditions)
- **Maximum Throughput:** One instruction per clock cycle (CPI = 1)
- **Real Throughput:** $\frac{1}{T \times (1 + \text{Stall Cycles per Instruction})}$

10.2 Performance Comparison

Metric	Sequential Execution	Pipelined Execution
Time for 1 instruction	mT	mT (first instruction)
Time for N instructions	$N \times mT$	$(N + m - 1)T$
Throughput	$\frac{1}{mT}$	$\frac{1}{T}$ (ideal)
Speedup	1	$\frac{Nm}{N+m-1} \approx m$ (for large N)

11 Benefits and Disadvantages of Pipelining

11.1 Benefits of Pipelining

1. Increased Instruction Throughput:

- Multiple instructions processed simultaneously
- Higher instruction completion rate
- Better utilization of processor resources

2. Better Hardware Utilization:

- Different processor components work concurrently
- Reduced idle time for functional units
- More efficient use of existing hardware

3. Efficient for Repetitive Tasks:

- Ideal for streaming data applications
- Excellent for loop-intensive programs
- Consistent performance for similar operations

4. Cost-Effectiveness:

- Significant performance improvement
- Relatively small hardware overhead
- Better performance-to-cost ratio

11.2 Disadvantages of Pipelining

1. Complex Design:

- Sophisticated control logic required
- Complex hazard detection and resolution
- Difficult debugging and verification

2. Pipeline Hazards:

- Structural, data, and control hazards
- Performance degradation due to stalls
- Complex hazard resolution mechanisms needed

3. **Branch Instruction Problems:**

- Control hazards increase with pipeline depth
- Branch misprediction penalties
- Complex branch prediction mechanisms required

4. **Unpredictable Performance:**

- Throughput depends on instruction mix
- Performance varies with program characteristics
- Difficult to guarantee consistent performance

5. **Additional Hardware Overhead:**

- Pipeline registers between stages
- Forwarding logic for hazard resolution
- Control circuitry for pipeline management

11.3 When to Use Pipelining

Ideal Scenarios:

- Regular, repetitive computations
- Applications with predictable control flow
- Programs with minimal data dependencies
- Streaming data processing

Challenging Scenarios:

- Irregular control flow with many branches
- High data dependency between instructions
- Varying execution times for different operations
- Resource-constrained environments